

Documentazione Software Architecture Design

Music Player

A cura di :
Ammendola Edoardo
Castellano Bruno
Cerra Claudia
Finamore Francesco

Indice

1) UserStories

- Priorità alta
- Priorità media
- Priorità bassa

2) Definition of Done

3) Architettura Software

4) Diagramma dei Package

5) Diagramma delle classi

- State Pattern
- Observer Pattern
- Command Pattern
- Strategy Pattern
- Iterator Pattern
- Singleton Pattern

6) Sprint Backlog - 1

7) Scomposizione in Task tecnici

1) UserStories

● Alta Priorità

US1 (2 SP) : Come utente, voglio che ogni brano aggiunto alla mia libreria venga salvato permanentemente nello stesso istante in cui lo inserisco, per non perdere i dati in caso di chiusura improvvisa.

- **AC 1 (Salvataggio):**
 - **Given:** L'utente si trova nel modulo di aggiunta brano e ha inserito dati validi.
 - **When:** L'utente conferma l'inserimento.
 - **Then:** Il sistema registra immediatamente il brano nella memoria non volatile, rendendolo persistente e al sicuro da eventuali interruzioni dell'app.
- **AC 2 (Caricamento libreria al riavvio):**
 - **Given:** Nel sistema sono presenti brani memorizzati in una sessione precedente.
 - **When:** L'applicazione viene avviata.
 - **Then:** Il sistema recupera tutti i brani salvati e popola automaticamente la visualizzazione della libreria generale.

US2 (2 SP) : Come utente, voglio che le playlist create vengano salvate permanentemente e che rimangano sempre coerenti con la libreria principale, così da non doverle ricreare.

- **AC 1 (Salvataggio):**
 - **Given:** L'utente ha creato una playlist o vi ha aggiunto/rimosso un brano.
 - **When:** L'azione di modifica della playlist viene confermata.
 - **Then:** Il sistema aggiorna istantaneamente la memoria registrando il nome della playlist e l'esatto elenco dei brani.
- **AC 2 (Caricamento playlist al riavvio):**
 - **Given:** Nel sistema sono presenti playlist memorizzate.
 - **When:** L'applicazione viene avviata.
 - **Then:** Il sistema recupera le playlist e mostra i rispettivi brani nell'ordine corretto.
- **AC 3 (Sincronizzazione Eliminazione a cascata):**
 - **Given:** L'utente ha una o più playlist salvate che contengono il brano "X".
 - **When:** L'utente elimina definitivamente il brano "X" dalla libreria principale.
 - **Then:** Il sistema rimuove automaticamente il brano "X" da tutte le playlist che lo contenevano e aggiorna istantaneamente il salvataggio di tali playlist in memoria.

US3 (2 SP) : Come utente, voglio aggiungere una traccia inserendo titolo, autore, lunghezza, genere e anno di pubblicazione per popolare la libreria.

- **AC 1 (Inserimento valido):**
 - **Given:** L'utente si trova nel modulo di aggiunta traccia.
 - **When:** L'utente inserisce dati validi per titolo, autore, lunghezza, genere e anno, e conferma.
 - **Then:** L'applicazione salva la traccia e la rende disponibile all'interno della libreria generale.
- **AC 2 (Dati mancanti):**
 - **Given:** L'utente si trova nel modulo di aggiunta traccia.
 - **When:** L'utente lascia uno o più campi obbligatori vuoti e tenta di salvare.
 - **Then:** Il sistema blocca il salvataggio e mostra un messaggio di errore all'utente.

US4 (2 SP) : Come utente, voglio modificare i dettagli di una traccia.

- **AC 1 (Modifica confermata):**

- **Given:** L'utente ha selezionato una traccia specifica dalla libreria.
- **When:** L'utente modifica uno o più dettagli (es. cambia l'autore) e salva.
- **Then:** I dettagli della traccia vengono aggiornati e la modifica è visibile in tutta l'applicazione.

US5 (1 SP) : Come utente, voglio eliminare una traccia dalla libreria.

- **AC 1 (Eliminazione completa):**

- **Given:** L'utente ha selezionato una traccia nella libreria.
- **When:** L'utente esegue l'azione di eliminazione e conferma.
- **Then:** La traccia scompare dalla visualizzazione della libreria (e conseguentemente dalle playlist in cui era inserita).

US6 (2 SP) : Come utente, voglio vedere tutte le canzoni presenti all'interno della libreria in una user interface.

- **AC 1 (Visualizzazione lista popolata):**

- **Given:** La libreria contiene almeno una traccia salvata.
- **When:** L'utente accede alla sezione "Tracks".
- **Then:** Il sistema mostra l'elenco completo di tutte le tracce indipendentemente dalle playlist.

US7 (1 SP): Come utente, voglio creare una playlist vuota identificata da un nome univoco.

- **AC 1 (Creazione standard):**

- **Given:** L'utente si trova nella sezione playlist.
- **When:** L'utente crea una nuova playlist inserendo un nome univoco.
- **Then:** Il sistema genera una playlist vuota col nome univoco scelto in precedenza e la aggiunge all'elenco.

- **AC 2 (Controllo univocità):**

- **Given:** Esiste già una playlist chiamata con il nome scelto.
- **When:** L'utente tenta di creare una nuova playlist con il nome scelto.
- **Then:** Il sistema rifiuta la creazione e avvisa l'utente che il nome è già in uso.

US8 (2 SP): Come utente, voglio aggiungere una traccia dalla libreria a una playlist esistente.

- **AC 1 (Aggiunta traccia):**

- **Given:** L'utente ha selezionato una playlist esistente.
- **When:** L'utente seleziona una traccia dalla libreria e la aggiunge alla playlist.
- **Then:** La traccia compare all'interno della playlist selezionata.

- **AC 2 (Traccia già presente nella playlist):**

- **Given:** L'utente ha selezionato una playlist esistente.
- **When:** L'utente seleziona una traccia dalla libreria e cerca di aggiungerla alla playlist mentre la playlist già contiene la traccia scelta.
- **Then:** Il sistema impedisce la modifica e mostra un messaggio di errore all'utente.

US9 (1 SP): Come utente, voglio rimuovere una traccia specifica da una playlist.

- **AC 1 (Rimozione traccia):**

- **Given:** L'utente ha selezionato una playlist esistente.
- **When:** L'utente seleziona una traccia all'interno della playlist per rimuoverla.
- **Then:** La traccia scompare dalla playlist selezionata, ma non viene rimossa dalla libreria.

US10 (3 SP): Come utente, voglio che le aggiunte e rimozioni sulle playlist si riflettano istantaneamente sulla coda di riproduzione, senza interrompere la canzone attualmente in esecuzione.

- **AC 1:**

- **Given:** L'utente ha selezionato una playlist esistente.
- **When:** L'utente decide di rimuovere o aggiungere una traccia mentre la playlist selezionata è in riproduzione,
- **Then:** La coda del riproduttore viene aggiornata, senza interrompere la canzone attualmente in esecuzione.

US11 (1 SP): Come utente, voglio modificare il nome di una playlist.

- **AC 1 (Modifica nome univoco):**

- **Given:** L'utente ha selezionato una playlist esistente.
- **When:** L'utente decide di modificare il nome della playlist.
- **Then:** Il nome della playlist viene modificato.

- **AC 2 (Controllo univocità playlist):**

- **Given:** Esiste già una playlist chiamata con il nome scelto.
- **When:** L'utente tenta di modificare il nome di una playlist con il nome scelto.
- **Then:** Il sistema rifiuta la modifica e avvisa l'utente che il nome è già in uso.

US12 (1 SP) : Come utente, voglio cancellare una playlist senza eliminare le tracce dalla libreria ma solo dalla playlist.

- **AC 1 (Cancellazione sicura):**

- **Given:** L'utente ha selezionato una playlist contenente alcune tracce.
- **When:** L'utente elimina la playlist.
- **Then:** La playlist scompare dal sistema, ma le tracce rimangono disponibili nella libreria generale.

US13 (2 SP) : Come utente, voglio vedere graficamente tutte le tracce presenti all'interno di una playlist.

- **AC 1 (Esplorazione playlist):**

- **Given:** L'utente visualizza l'elenco delle proprie playlist.
- **When:** L'utente clicca su una specifica playlist.
- **Then:** Il sistema mostra esclusivamente le tracce contenute in quella specifica playlist, nel loro ordine di inserimento.

US14 (2 SP) : Come utente, voglio vedere graficamente tutte le playlist esistenti in un apposito elenco.

- **AC 1 (Visualizzazione lista popolata):**

- **Given:** Esiste almeno una playlist.
- **When:** L'utente accede alla sezione "Playlist".
- **Then:** Il sistema mostra a schermo l'elenco completo di tutte le playlist.

US15 (3 SP) : Come utente, voglio avviare (Play) una traccia della mia libreria caricando tutti i brani successivi nella coda di riproduzione.

- **AC 1 (Riproduzione sequenziale dalla Libreria):**

- **Given:** L'utente si trova nella schermata della Libreria Musicale con un elenco di brani visibili.
- **When:** L'utente seleziona e avvia un brano specifico dall'elenco.

- **Then:** Il sistema avvia la riproduzione del brano scelto.

US16 (3 SP) : Come utente, voglio mettere in pausa (Pause) una traccia della mia playlist

- **AC 1 (Messa in pausa di una traccia):**

- **Given:** Il sistema sta riproducendo un brano musicale.
- **When:** L'utente seleziona l'icona di "Pausa".
- **Then:** Il sistema congela la barra di avanzamento del tempo sul secondo esatto dell'interruzione.

- **AC 2 (Ripresa della riproduzione dalla pausa):**

- **Given:** Il sistema ha un brano attualmente in stato di pausa.
- **When:** L'utente seleziona nuovamente l'icona di "Play".
- **Then:** Il sistema riattiva l'avanzamento della barra del tempo dal punto in cui era stato interrotta.

US17 (3 SP) : Come utente, voglio andare alla traccia successiva (Skip) della mia playlist

- **AC 1 (Avanzamento al brano successivo):**

- **Given:** Il sistema sta riproducendo un brano musicale all'interno di una playlist (o della libreria) e nella coda sono presenti altre canzoni successive.
- **When:** L'utente seleziona l'icona di "Skip".
- **Then:** Il sistema azzerava il timer, carica la traccia successiva rispettando l'ordine della coda e ne avvia la riproduzione in automatico, aggiornando i dettagli del brano sulla UI.

- **AC 2 (Tentativo di skip sull'ultimo brano della coda):**

- **Given:** Il sistema sta riproducendo l'ultimo brano disponibile nella coda di riproduzione corrente.
- **When:** L'utente seleziona l'icona di "Skip".
- **Then:** Il sistema interrompe la riproduzione del brano corrente e, non essendoci ulteriori tracce successive, si arresta azzerando l'interfaccia, a meno che non siano attive modalità di ripetizione (Loop).

US18 (3 SP) : Come utente, voglio avviare (Play) una playlist caricandola per intero nella coda di riproduzione.

- **AC 1 (Riproduzione playlist non vuota):**

- **Given:** L'utente si trova nella schermata Playlist.
- **When:** L'utente seleziona una playlist e l'avvia.
- **Then:** Il sistema avvia la riproduzione del primo brano della playlist scelta.

US19 (3 SP) : Come utente, voglio poter saltare la playlist attualmente in riproduzione, affinché il sistema passi direttamente al primo brano della playlist successiva nell'elenco o nella coda complessiva.

- **AC 1 (Passaggio alla playlist successiva):**

- **Given:** Il sistema sta riproducendo un brano di una determinata playlist e nell'elenco è presente almeno un'altra playlist successiva.
- **When:** L'utente seleziona il comando "Skip"
- **Then:** Il sistema interrompe immediatamente la riproduzione corrente, carica i brani della nuova playlist, imposta il primo brano di quest'ultima come corrente e ne avvia la riproduzione, aggiornando la UI con i dati della nuova playlist.

- **AC 2 (Tentativo di skip sull'ultima playlist dell'elenco):**

- **Given:** Il sistema sta riproducendo l'ultima playlist disponibile nell'elenco o nella coda complessiva delle playlist.
- **When:** L'utente seleziona il comando "Skip".

- **Then:** Il sistema interrompe la riproduzione del brano corrente e, non essendoci ulteriori playlist successive da caricare, si arresta e azzerà il timer.

US20 (1 SP): Come utente, voglio poter scegliere la modalità di riproduzione Sequenziale.

- **AC 1 (Modalità Sequenziale):**

- **Given:** La modalità "Sequenziale" è attiva e una traccia termina.
- **When:** Il sistema deve decidere il brano successivo.
- **Then:** Viene riprodotta la traccia esattamente successiva nell'ordine della playlist.

US21 (2 SP): Come utente, voglio poter scegliere la modalità di riproduzione Shuffle.

- **AC 1 (Modalità Shuffle):**

- **Given:** La modalità "Shuffle" è attiva.
- **When:** Una traccia termina o l'utente preme "Skip".
- **Then:** Il sistema seleziona casualmente il brano successivo tra quelli non ancora riprodotti nella playlist corrente.

US22 (2 SP): Come utente, voglio poter scegliere la modalità di riproduzione Loop.

- **AC 1 (Modalità Loop):**

- **Given:** La modalità "Loop" è attiva su una traccia.
- **When:** La traccia giunge al termine.
- **Then:** Il sistema la fa ripartire da capo.

US23 (5 SP): Come utente, voglio vedere graficamente l'aggiornamento delle tracce e delle playlists.

- **AC 1 (Avanzamento del tempo):**

- **Given:** Una traccia musicale è in riproduzione.
- **When:** Il tempo fisico scorre.
- **Then:** L'interfaccia aggiorna in tempo reale la barra di avanzamento e il contatore di tempo del brano.

- **AC 2 (Aggiornamento informazioni brano):**

- **Given:** La riproduzione passa in automatico da una traccia alla successiva.
- **When:** Inizia la nuova traccia.
- **Then:** L'interfaccia aggiorna dinamicamente i testi mostrati senza che l'utente debba cliccare nulla.

- **Priorità media**

US24 (8 SP): Come utente, voglio annullare la mia ultima operazione di aggiunta o rimozione da una playlist

- **AC 1(undo di un aggiunta):**

- **Given:** L'utente ha aggiunto una traccia alla libreria o ad una playlist
- **When:** l'utente clicca sul bottone "Undo"
- **Then:** il sistema rimuove la traccia aggiunta

- **AC 2 (undo di una rimozione):**

- **Given:** l'utente ha appena eliminato una traccia da una playlist o dalla libreria
- **When:** l'utente clicca sul bottone "Undo"
- **Then:** il sistema inserisce nuovamente la traccia nella playlist o dalla libreria, nella stessa posizione in cui si trovava prima

US25 (3 SP): Come utente, voglio vedere nella home page le tracce e le playlist più riprodotte

- **AC 1 (Visualizzazione classifiche aggiornate):**
 - **Given:** L'applicazione ha tracciato lo storico degli ascolti dell'utente.
 - **When:** L'utente accede alla schermata principale.
 - **Then:** L'elenco generale delle tracce e delle playlist si presenta ordinando automaticamente verso l'alto i contenuti riprodotti più di frequente.

US26 (5 SP): Come utente voglio che l'applicazione possa creare delle playlist automatiche basate sul genere o anno di uscita

- **AC 1 (Creazione per Genere):**
 - **Given:** La libreria contiene brani appartenenti a diversi generi musicali.
 - **When:** L'utente richiede la creazione di una playlist automatica selezionando un filtro relativo al genere.
 - **Then:** Il sistema genera una nuova playlist popolata esclusivamente con tutti i brani del genere scelto.
- **AC 2 (Creazione per Anno):**
 - **Given:** La libreria generale contiene tracce con diversi anni di pubblicazione.
 - **When:** L'utente richiede la creazione di una playlist automatica selezionando un filtro relativo all'anno di uscita.
 - **Then:** Il sistema genera una nuova playlist contenente solo le tracce pubblicate nell'anno richiesto.

US27 (5 SP): Come utente voglio assegnare Tag visivi alle tracce (favourite, explicit, new release) per poter creare playlist automaticamente tramite i tag scelti.

- **AC 1 (Assegnazione e visualizzazione):**
 - **Given:** L'utente sta visualizzando i dettagli di una traccia o la lista della libreria.
 - **When:** L'utente assegna una o più tag a una specifica traccia.
 - **Then:** Il sistema aggiorna l'interfaccia e mostra il relativo indicatore visivo.
- **AC 2 (Playlist da Tag):**
 - **Given:** Diverse tracce in libreria sono state contrassegnate con un tag.
 - **When:** L'utente decide di generare una playlist automatica basata sullo specifico tag.
 - **Then:** Il sistema raggruppa e inserisce nella nuova playlist solo le canzoni che possiedono il tag richiesto.

- **Priorità bassa**

US28 (3 SP) : Come utente, voglio poter cambiare l'ordine dei brani all'interno di una playlist tramite l'interfaccia, per personalizzare la mia playlist

- **AC 1 (Spostamento e salvataggio):**
 - **Given:** L'utente visualizza l'elenco dei brani all'interno di una specifica playlist.
 - **When:** L'utente sposta una traccia da una posizione a un'altra.
 - **Then:** La vista della lista si aggiorna mostrando il nuovo ordine, e il sistema salva le nuove posizioni.

US29 (3 SP) : Come utente, voglio riordinare i brani nella playlist anche durante la riproduzione di quella stessa playlist, cambiando anche la coda di riproduzione in corso.

- **AC 1 (Adattamento del motore):**

- **Given:** L'utente sta ascoltando il brano X della playlist y (esempio in posizione 2).
- **When:** L'utente sposta un altro brano portandolo dalla posizione 5 alla 1 (facendo scalare il brano X alla nuova posizione 3).
- **Then:** La musica del brano X non si interrompe aggiornando automaticamente la coda di riproduzione

2) Definition of Done

Una User Story si considera “Done” solo quando soddisfa tutti i seguenti requisiti :

- **Qualità del codice e architettura**

- Le interfacce, i metodi principali e l'uso specifico dei Design Pattern sono adeguatamente commentati per facilitarne la comprensione.

- **Testing**

- Sono stati sviluppati Test Unitari automatizzati utilizzando il framework **JUnit**.
- I nuovi test (e quelli precedenti) vengono eseguiti con successo.

- **Gestione del versioning (GitHub)**

- Il codice è stato sottoposto a commit e push sul repository GitHub del progetto.
- L'integrazione sul branch principale è avvenuta solo dopo una revisione del codice (*Code - Review*) da parte di almeno un altro membro del team.
- L'applicazione compila ed esegue correttamente dal branch principale senza errori o merge conflicts.

- **Tracciamento (Trello)**

- I requisiti funzionali descritti nei *Criteri di Accettazione* specifici della User Story sono stati tutti soddisfatti.
- Tutte le task associate alla storia sulla bacheca Trello sono state completate e spostate nella colonna "Done".

3) Architettura Software

La scelta di un'architettura software prima dell'inizio dello sviluppo è fondamentale per comprendere il quadro generale del sistema, comunicare le decisioni di progettazione, distribuire il lavoro tra i membri del team e pianificare una strategia di testing efficace.

Trattandosi di un'applicazione desktop in Java con un'interfaccia grafica dinamica, abbiamo adottato il pattern architetturale **Model-View-Controller (MVC)**. Questa scelta sposa appieno il principio della **Separation of Concerns** (Separazione delle Responsabilità).

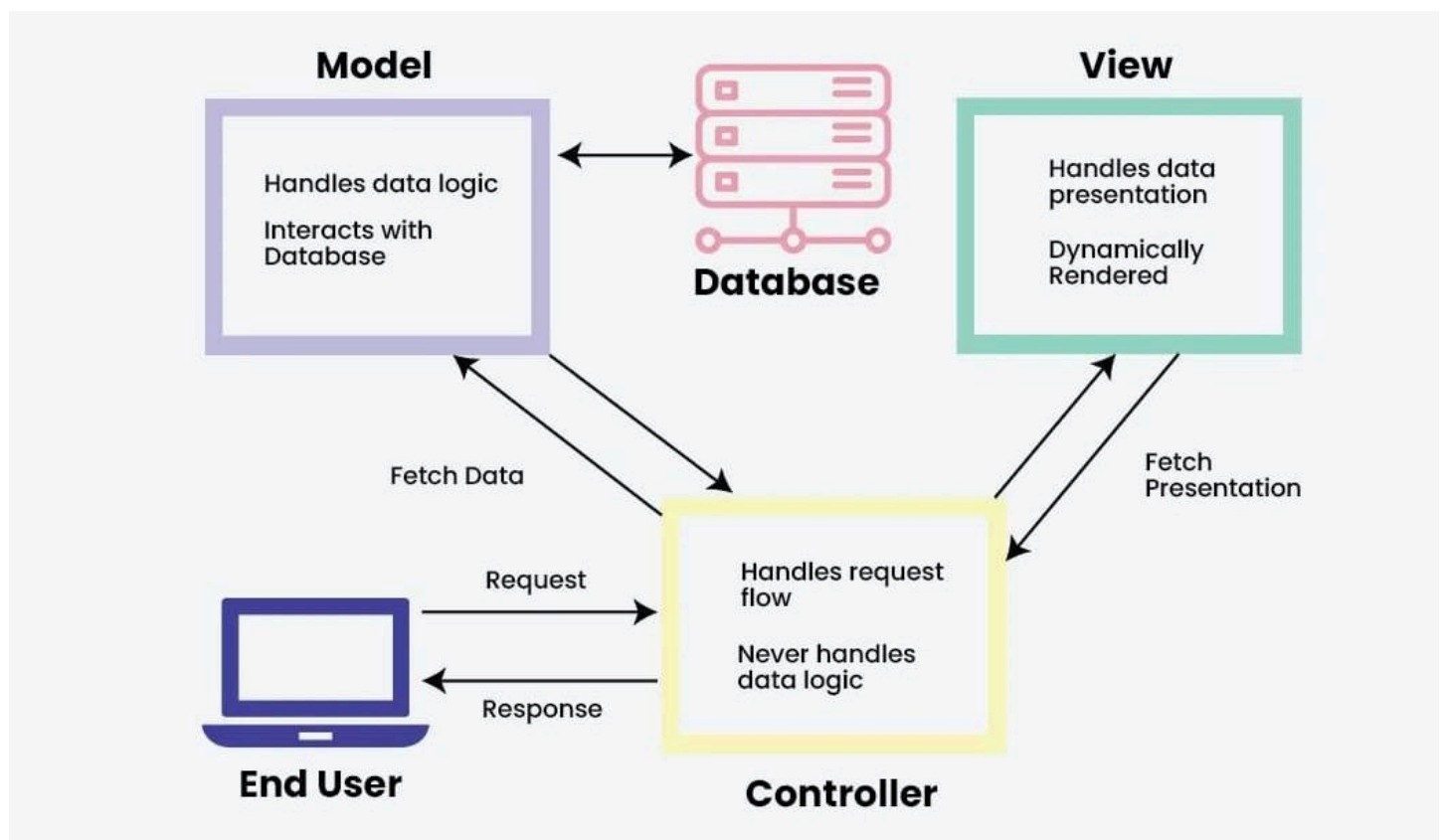
La **View**, realizzata tramite JavaFX, è strutturata internamente secondo il pattern **Composite**: l'interfaccia grafica è concepita come un albero gerarchico in cui elementi semplici (es. bottoni, etichette di testo) e contenitori complessi (es. pannelli, finestre) vengono trattati in modo uniforme, semplificando la costruzione e l'aggiornamento a cascata della UI.

Oltre a questo, la View si occupa esclusivamente del rendering visivo e delega al **Controller** la gestione degli eventi generati dall'utente (es. il click sul tasto Play), adottando il pattern **Strategy**. Abbiamo scelto un approccio in cui la View è disaccoppiata dal Model: è il Controller a fare da intermediario, garantendo una comunicazione tra i moduli basata sul **Data Coupling** (il livello più basso e ottimale di accoppiamento).

Il **Model** incapsula l'intera logica di business e mantiene il proprio stato. Per comunicare i cambiamenti (es. l'avanzamento del tempo di una traccia) in tempo reale senza creare dipendenze dirette, il Model implementa il pattern **Observer**: il Controller funge da *Observer* in ascolto e, una volta notificato, provvede ad aggiornare la View.

La persistenza dei dati (tracce e playlist) è gestita tramite un database locale relazionale **SQLite**. Questa integrazione è totalmente incapsulata all'interno del Model, mantenendo il resto dell'applicazione all'oscuro delle query SQL.

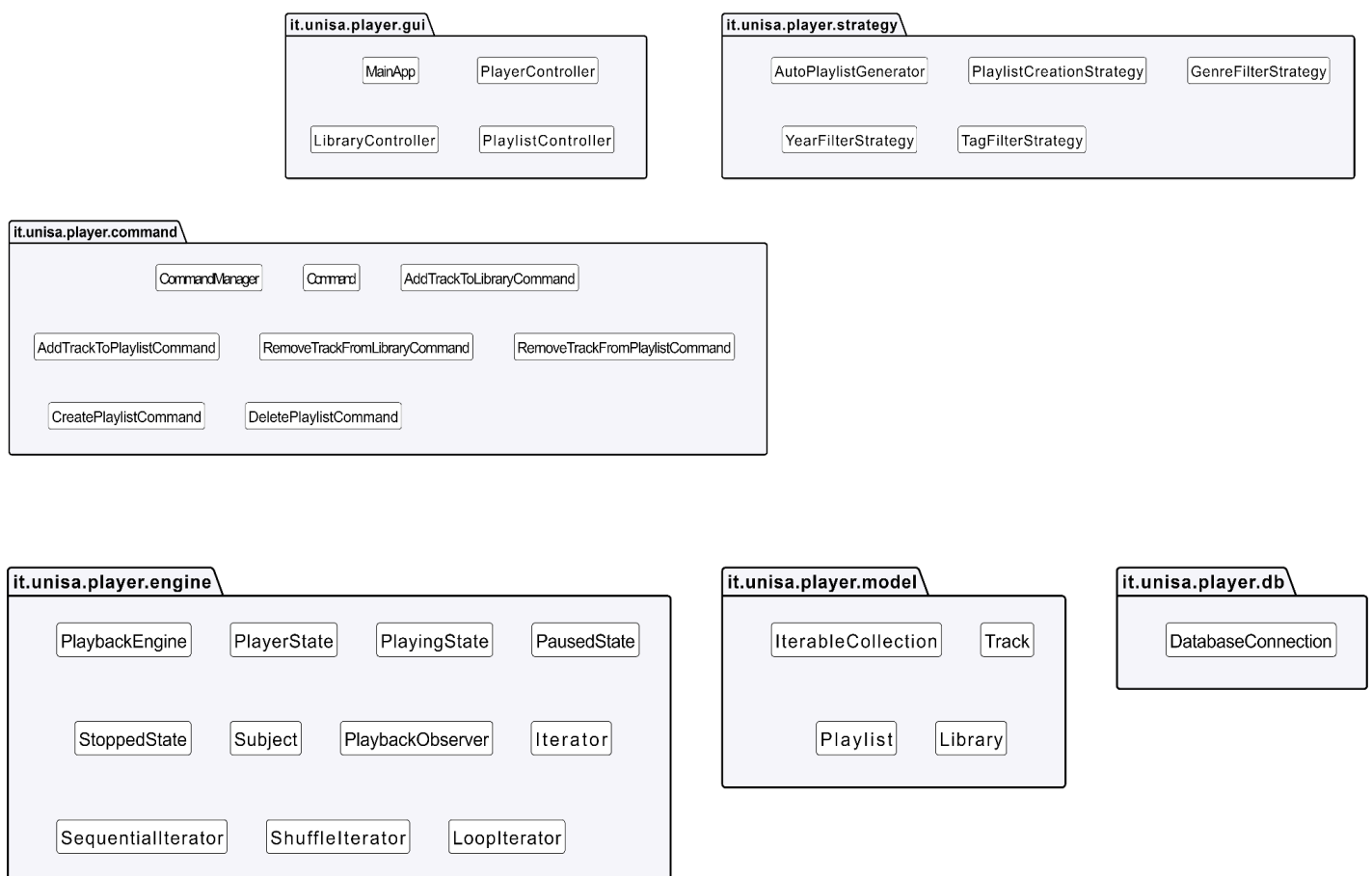
Questa architettura, orientata alla **Functional Cohesion** (Alta Coesione) all'interno dei singoli moduli, garantisce un sistema facilmente manutenibile ed estendibile, permettendo inoltre una forte testabilità isolata della logica di business tramite unit test con **JUnit**.



4) Diagramma dei Package

Un diagramma dei package illustra come le classi sono organizzate in package. Il software è diviso in sei package :

- **it.unisa.player.model** : rappresenta il cuore dell'applicazione. Incapsula lo stato del dominio (Tracce, Playlist) e le relative regole di business, garantendo l'integrità dei dati prima che vengano esposti agli altri layer del sistema.
- **it.unisa.player.db** : questo package gestisce la persistenza dell'applicazione. Contiene la classe che centralizza l'accesso al file locale SQLite. Isola il database dal resto del programma.
- **it.unisa.player.engine** : questo package gestisce il comportamento dinamico dell'app. Si interessa della logica di riproduzione (Stati e Iteratori).
- **it.unisa.player.command** : lo scopo principale di questo package è quello di tracciare lo storico delle operazioni distruttive o di aggiunta per permettere di annullarle.
- **it.unisa.player.strategy** : isola gli algoritmi complessi di ricerca, filtraggio e ordinamento dei dati. Consente di variare dinamicamente i criteri di selezione dei brani senza alterare le collezioni originali nel Model.
- **it.unisa.player.gui** : questo package contiene le classi di interfaccia utente e i file JavaFx (.fxml) che definiscono il layout. Non contiene logica di business né calcoli matematici.

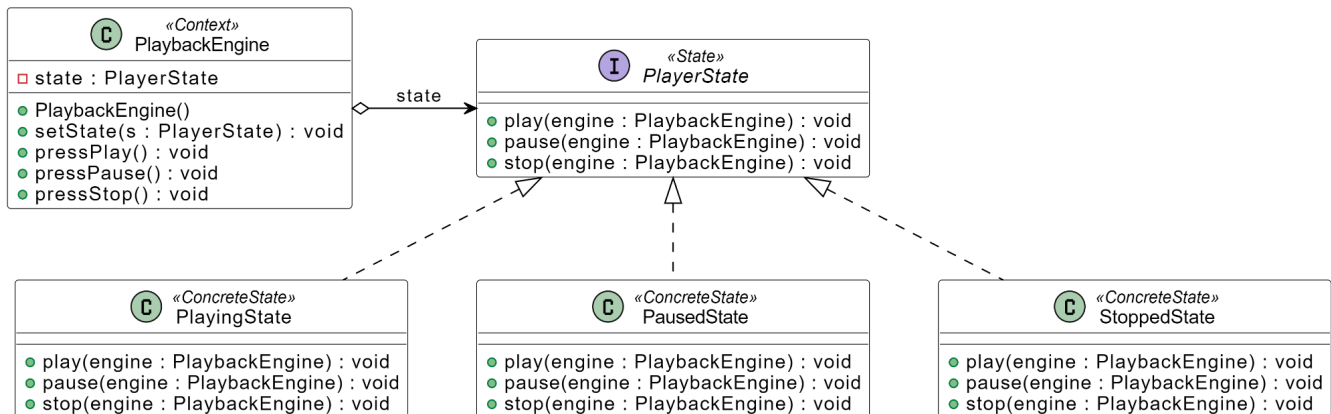


5) Diagramma delle classi

Per fornire una documentazione tecnica esaustiva senza compromettere la leggibilità, è stato scelto di scomporre il diagramma delle classi totale in diversi diagrammi delle classi di dettaglio per ogni pattern adottato.

• State Pattern

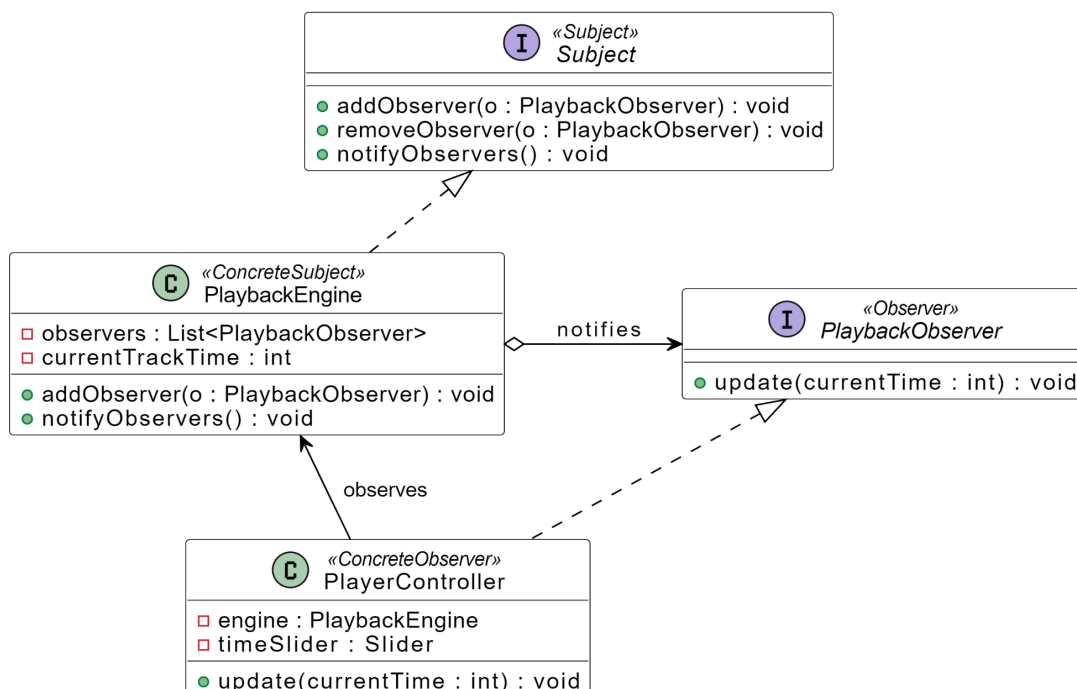
Attraverso questo pattern si vogliono gestire i continui cambi di stato del player (Play, Pausa, Stop). L'incapsulamento di ogni stato in una classe dedicata permette all'applicazione di variare il proprio comportamento a runtime.



2 - Diagramma delle classi: State Pattern

• Observer Pattern

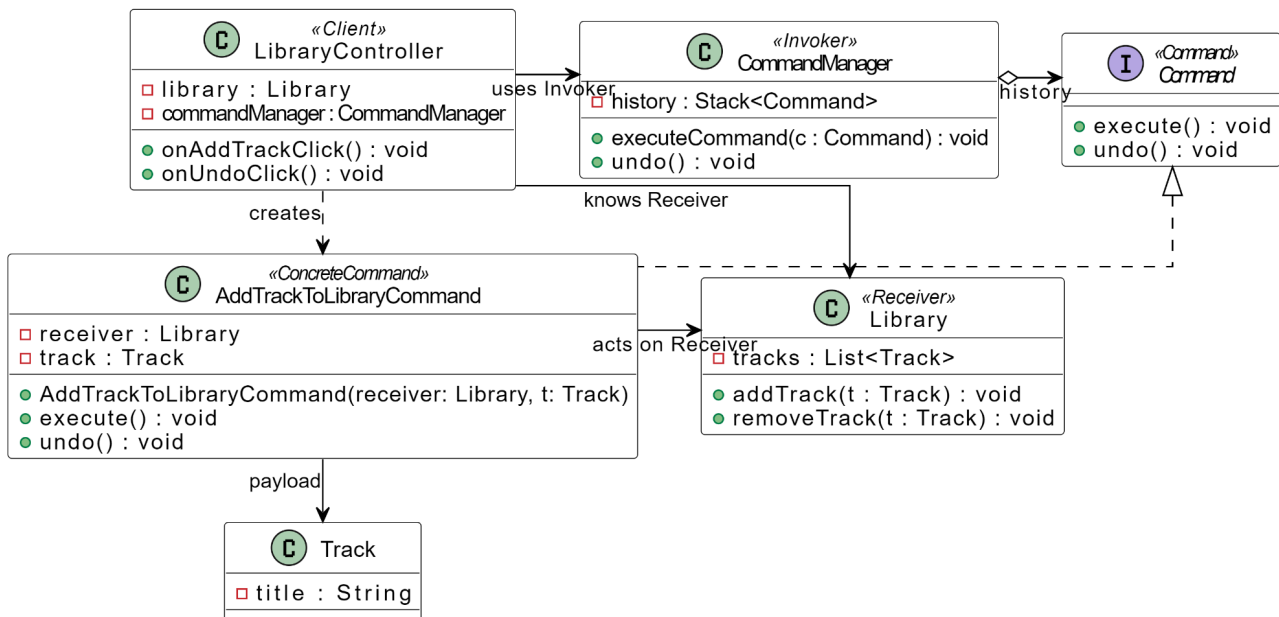
Attraverso questo pattern si risolve la necessità di aggiornare l'interfaccia grafica in tempo reale in risposta ai cambiamenti interni. Questo approccio permette di disaccoppiare le Subject dalla View garantendo un basso accoppiamento e rispettando il principio di Separation of Concerns (SoC)



3 - Diagramma delle classi: Observer Pattern

- **Command Pattern**

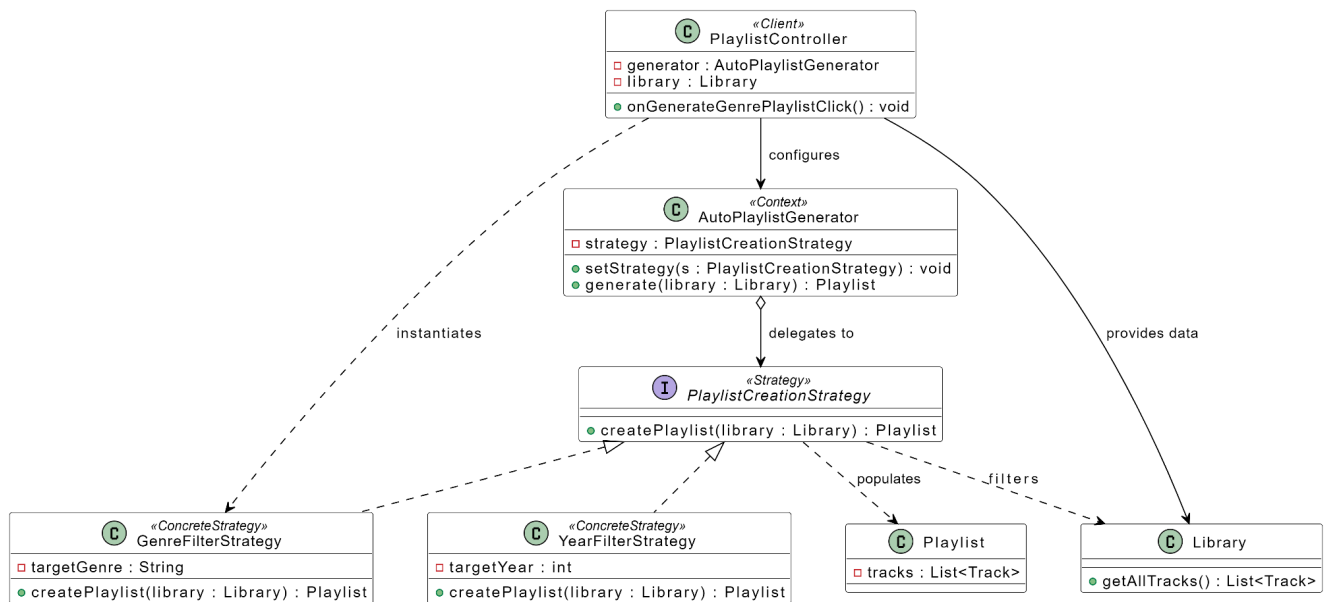
Attraverso questo pattern si vuole tracciare lo storico delle azioni dell'utente per permettere di mantenere una cronologia delle stesse e fornirgli una funzionalità di annullamento (Undo). Incapsulare ogni azione in un oggetto dedicato rispetta il Single Responsibility Principle (SRP), poiché ogni comando ha solo il compito di eseguire o annullare la propria operazione. Nel diagramma sottostante, per semplicità, è mostrato solo uno dei tanti *ConcreteCommand*.



4 - Diagramma delle classi: Command Pattern

- **Strategy Pattern**

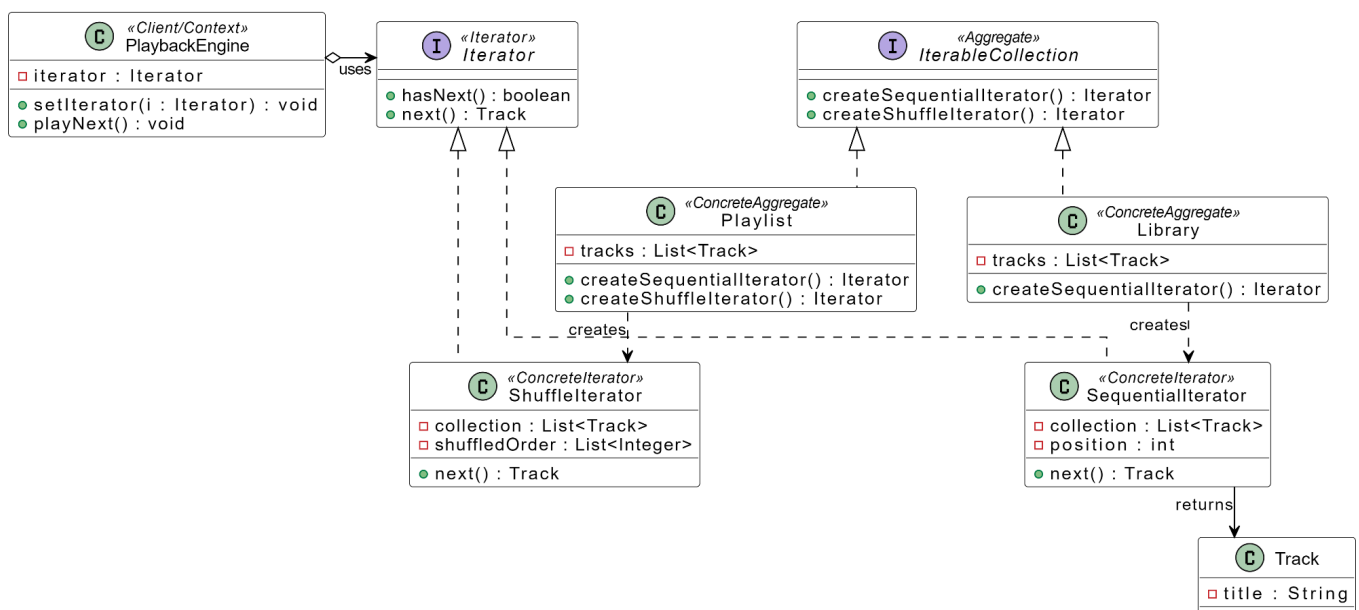
Attraverso questo pattern si vuole gestire la creazione dinamica delle playlist automatiche. Lo scopo è quello di isolare gli algoritmi di filtraggio in classi separate per facilitare l'aggiunta futura di criteri di ricerca senza modificare il resto del codice, rispettando l'Open/Closed Principle.



5 - Diagramma delle classi: Strategy Pattern

• Iterator Pattern

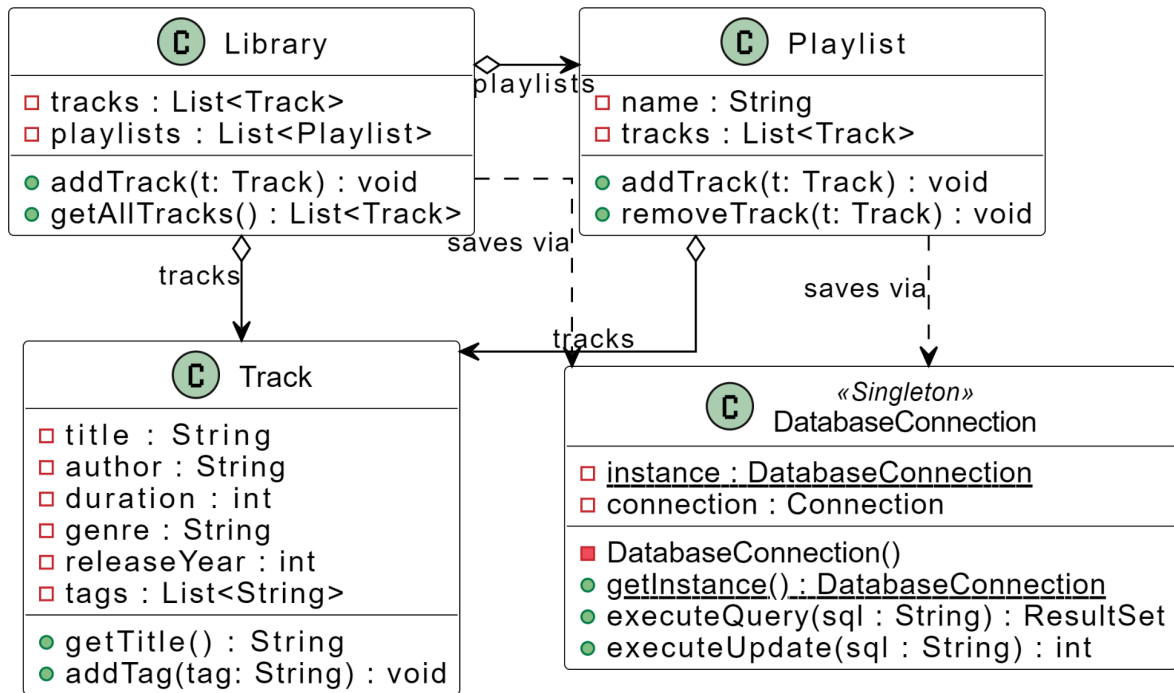
Attraverso questo pattern si vuole gestire l'iterazione dei brani presenti all'interno delle code musicali. Si vuole permettere di passare al brano successivo in diverse modalità (Sequenziale, Shuffle, Loop) delegando la logica all'iteratore e favorendo l'Information Hiding.



6 - Diagramma delle classi: Iterator Pattern

• Singleton Pattern

Attraverso questo pattern si centralizza l'accesso al database locale SQLite. In questo modo l'applicazione instancierà una singola volta una connessione e tutte le successive volte verrà utilizzata la stessa connessione per il salvataggio dei dati.



7 - Diagramma delle classi: Singleton Pattern

6) Sprint Backlog - 1

Per lo Sprint 1 abbiamo scelto le seguenti User Stories ,per un totale di 21 SP, e le abbiamo suddivise in tre fasi macro-logiche :

- **Fase 1 (Gestione della Persistenza):**
 - **US1 (2 SP):** Salvataggio brano DB
 - **US2 (2 SP):** Salvataggio playlist DB
- **Fase 2 (Gestione della libreria):**
 - **US6 (2 SP):** Visualizzazione libreria
 - **US3 (2 SP):** Aggiunta traccia
 - **US5 (1 SP):** Eliminazione traccia
 - **US4 (2 SP):** Modifica dettagli di una traccia
- **Fase 3 (Gestione della Playlist):**
 - **US14 (2 SP):** Visualizzazione elenco Playlist
 - **US7 (1 SP):** Creazione Playlist vuota
 - **US13 (2 SP):** Visualizzazione tracce di una Playlist
 - **US8 (2 SP):** Aggiunta traccia a Playlist
 - **US9 (1 SP):** Rimozione traccia da Playlist
 - **US11 (1 SP):** Modifica nome playlist
 - **US12 (1 SP):** Cancellazione playlist

7) Scomposizione in Task Tecnici

Per tradurre le User Stories in codice reale, abbiamo frammentato ogni storia in Task Tecnici. Questa scomposizione permette di tracciare con precisione le attività di sviluppo definendo chiaramente i criteri operativi necessari a soddisfare la *Definition of Done* di ogni singola funzionalità.

Fase 1 :

US1 (2 SP): Salvataggio brano DB

- Task 1.1 : Aggiungere il driver JDBC SQLite al progetto.
- Task 1.2 : Implementare la classe DatabaseConnection applicando il Singleton Pattern con costruttore privato e getInstance().
- Task 1.3 : Creare il metodo di inizializzazione nel Singleton per eseguire la query CREATE TABLE IF NOT EXISTS Tracce.
- Task 1.4 : Implementare la classe Model Track con tutti i suoi attributi.
- Task 1.5 : Scrivere un Test JUnit per verificare che DatabaseConnection.getInstance() non restituisca eccezioni.

US2 (2 SP): Salvataggio playlist DB

- Task 2.1 : Aggiungere al Singleton le query per creare la tabella Playlists e la tabella associativa Playlist_Tracce.
- Task 2.2 : Implementare la classe Model Playlist contenente una List<Track>.

Fase 2 :

US6 (2 SP): Visualizzazione libreria

- Task 6.1 : Creare il file LibraryView.fxml con una TableView configurata per i campi della traccia.
- Task 6.2 : Implementare la classe LibraryController definendo gli @FXML fields.
- Task 6.3 : Implementare il metodo getAllTracks() nella classe Library che interroga il DB tramite il Singleton e restituisce la lista al Controller per popolare la UI.

US3 (2 SP): Aggiunta traccia

- Task 3.1 : Creare l'interfaccia Command e la classe CommandManager con il suo Stack.
- Task 3.2 : Implementare AddTrackToLibraryCommand che riceve la Library come Receiver.
- Task 3.3 : Implementare in Library il metodo addTrack(Track t) , che lancia executeUpdate.
- Task 3.4 : Aggiungere in LibraryController l'evento onAddTrackClick() che istanzia il comando e lo passa al CommandManager.
- Task 3.5 : Scrivere un Test JUnit per l'esecuzione corretta di AddTrackToLibraryCommand.

US4 (2 SP): Modifica dettagli di una traccia

- Task 4.1 : Implementare nella classe Library il metodo updateTrack(Track updatedTrack) per eseguire la query SQL di UPDATE su SQLite.

- Task 4.2 : Gestire l'evento del tasto "Modifica" nel `LibraryController`, verificando la selezione nella `TableView` e aprendo il modulo grafico precompilato.
- Task 4.3 : Implementare la logica del tasto "Salva", con validazione dei campi obbligatori, istanziamento del comando e invio al `CommandManager`.
- Task 4.4 : Aggiornare la UI eseguendo il refresh della `TableView` a valle del salvataggio, notificando il cambiamento visivo.
- Task 4.5 : Scrivere un test unitario JUnit per verificare il corretto funzionamento di `UpdateTrackCommand`.

Fase 3 :

US14 (2 SP): Visualizzazione elenco Playlist

- Task 14.1 : Creare il file `PlaylistView.fxml` con una `ListView` per i nomi delle playlist.
- Task 14.2 : Implementare `PlaylistController`.
- Task 14.3 : Aggiungere a `Library` una lista di playlist e il metodo per caricarle dal DB all'avvio.

US7 (1 SP): Creazione Playlist vuota

- Task 7.1 : Implementare `CreatePlaylistCommand` che riceve la `Library` e la nuova Playlist.
- Task 7.2 : Aggiornare `PlaylistController` con il campo di testo per il nome e l'invio del comando.

US13 (2 SP): Visualizzazione tracce di una Playlist

- Task 13.1 : Implementare la query JOIN nel metodo di caricamento della classe `Playlist` per recuperare i brani associati dal DB.
- Task 13.2 : Nel `PlaylistController`, aggiungere un listener alla `ListView`: al click, passare i brani della playlist selezionata alla `TableView` centrale.

US8 (2 SP): Aggiunta traccia a Playlist

- Task 8.1 : Implementare `AddTrackToPlaylistCommand`.
- Task 8.2 : Aggiungere in `Playlist` il metodo `addTrack(Track t)` che salva l'associazione in RAM e nel DB.
- Task 8.3 : Implementare nella UI un meccanismo per far invocare `onAddTrackToPlaylistClick()`.

US9 (1 SP): Rimozione traccia da Playlist

- Task 9.1 : Creare il comando `RemoveTrackFromPlaylistCommand`.
- Task 9.2 : Aggiungere in `Playlist` il metodo `removeTrack(Track t)` che rimuove la traccia dalla lista e lancia la query DELETE usando il Singleton.
- Task 9.3 : Nel `PlaylistController`, aggiungere un'opzione UI che agisce sulla `TableView` visibile in quel momento.

US11 (1 SP): Modifica nome playlist

- Task 11.1 : Implementare nella classe `Playlist` il metodo `rename(String newName)` per eseguire la UPDATE su SQLite.
- Task 11.2 : Configurare i componenti grafici nel `PlaylistController` per abilitare l'azione di rinomina.

- Task 11.3: Sviluppare la logica di gestione dell'evento di rinomina, intercettando l'eventuale eccezione di nome duplicato per mostrare un alert di errore sulla UI.
- Task 11.4: Scrivere i test unitari JUnit per verificare la corretta rinomina nel Model e il sollevamento dell'eccezione in caso di nome già esistente.

US12 (1 SP): Cancellazione playlist

- Task 12.1 : Creare il comando DeletePlaylistCommand.
- Task 12.2 : Aggiungere in Library il metodo removePlaylist(Playlist p) che fa anche la DELETE sul database
- Task 12.3 : Nel PlaylistController, aggiungere il bottone "Elimina", inserire un alert di conferma e inviare il comando in caso di risposta affermativa.